GSoC 2027 Proposal (Initial Draft)

Shared Synchronization and Channel Estimation for OFDM-Based Integrated Sensing and Communication in GNU Radio

Akshita Muchhal, Ahmedabad University, GitHub: MuchhalAkshita

Draft v0.1, November 1, 2026

This is an initial draft for M3 review. GNU Radio taking part in GSoC 2027, the project size and the official dates are not confirmed yet. The plan will be aligned to them once they are announced.

1 Abstract

Integrated sensing and communication (ISAC) lets one OFDM waveform carry data and also act as a radar signal. GNU Radio already has the two halves: OFDM transceiver blocks in gr-digital and OFDM radar processing in the gr-radar module. They were built under different assumptions and do not share synchronization or channel information. Communication synchronization tries to remove timing and frequency offsets. Radar needs the propagation delay that synchronization removes.

This project builds a way for both functions to work from one shared estimate of frame timing and channel state taken from the same received samples, in a way that keeps the delay information radar needs. It delivers a tested GNU Radio implementation, example flowgraphs, and a quantitative evaluation of synchronization error, channel-estimation error, BER and range error against SNR and delay.

2 Problem and motivation

ISAC is a topic in 6G research because it reduces spectrum and hardware duplication. OFDM is a natural waveform for it. Reciprocal filtering against known transmitted symbols is an established way to get range and Doppler from an OFDM signal (Sturm and Wiesbeck, 2011).

In GNU Radio, the communication side (gr-digital) estimates and removes timing and carrier frequency offsets so data can be decoded. The radar side (gr-radar) assumes frame timing is already established and is not designed to work with the gr-digital synchronization or channel-estimation blocks. Reusing the output of one chain for the other can erase information the other needs. The surveys and tutorials I reviewed in M1 (Liu et al., Qaisar et al., Dai et al., Vaezi et al.) describe robustness of a shared estimate as an open problem, and none of them implements or evaluates a solution. The closest GNU Radio work (Doly et al., 2023) is a hardware demonstration adapted from communication blocks, not a systematic quantitative study.

3 Goals and scope

3.1 Goals

- Analyze how the gr-digital OFDM synchronization and channel-estimation blocks and the gr-radar range and Doppler blocks make different assumptions about frame timing and channel state, and document where they diverge.
- Design and implement one concrete mechanism that lets both functions use a shared timing and channel estimate while preserving the delay the radar measurement needs.
- Provide unit and integration tests and at least one example flowgraph that runs the shared chain end to end in simulation.
- Evaluate the result against the unmodified behavior with synchronization error, channel-estimation error (NMSE), BER and range error across SNR and target delay.
- Submit the work upstream or to the appropriate module following the GNU Radio contribution process, and document it.

3.2 Out of scope

Multi-user and MIMO ISAC, standardized waveforms such as 5G NR numerology, and hardware-in-the-loop validation beyond an optional SDR demonstration, as in the M1 scope.

4 Technical approach

4.1 Baseline analysis

I will start by reading the gr-digital OFDM receive path and the gr-radar OFDM processing code and tracing exactly which timing, frequency-offset and channel quantities each one computes, uses and discards. I will check the exact block names and interfaces in the code before writing the design, because this draft does not claim them. The output is a short written comparison and a set of simulation flowgraphs that reproduce the incompatibility with known offsets and delays.

4.2 Shared estimate (hypothesis to test)

My working hypothesis is that the communication synchronizer can keep the frame-timing and delay information it estimates instead of discarding it, for example by carrying it alongside the stream as metadata, so that the radar stage can add it back when it computes range. The same idea applies to the channel estimate: the communication receiver uses it to equalize, and the radar stage needs the delay-related part of it. Whether this works, and whether the right place is gr-digital, gr-radar or a small new module, is something the baseline analysis and talks with maintainers must settle. I will propose one design to the community before writing the implementation.

4.3 Implementation and tests

- Implement the chosen mechanism as GNU Radio blocks or block changes, with Python and C++ QA tests following the project conventions.

- Build an example flowgraph: one OFDM transmitter, a simulated channel with noise, multipath and delay, and a receiver that produces both decoded data and a range estimate from the same samples.
- Keep changes small and reviewable, one pull request per logical change, each with tests and a filled-in pull request template.

4.4 Evaluation

Table 1: Evaluation metrics and how each is measured.

Metric	Measured as	Swept over
Synchronization error	Estimated minus true timing and frequency offset	SNR, true delay
Channel-estimation error	NMSE between estimated and true channel	SNR, multipath profile
Bit error rate	Decoded bits against transmitted bits	SNR
Range error	Estimated minus true range from the radar stage	SNR, target delay

Each metric is compared for the unmodified chain and the shared-estimate chain, and against the Doly et al. approach where it can be reproduced.

5 Deliverables

- Written analysis of the gr-digital and gr-radar assumption mismatch.
- Implementation of the shared-estimate mechanism with unit and integration tests.
- Example flowgraph and documentation for running it.
- Evaluation results with the four metrics, reproducible from scripts in the repository.
- One or more pull requests to the upstream project, with the state of each documented honestly.

6 Timeline

The plan below uses a 12 week layout for now. Project size, number of weeks and the official GSoC 2027 dates will be set once they are announced.

Table 2: Draft timeline.

Period	Work
Community bonding	Discuss the problem and the design hypothesis with GNU Radio maintainers; agree which module the work belongs in; read the contribution guidelines and templates again.
Weeks 1 to 2	Baseline analysis, reproduction flowgraphs, written comparison.
Weeks 3 to 4	Design proposal posted for maintainer feedback; prototype of the shared estimate in simulation.
Midterm	Working prototype with first BER and range measurements; design settled with maintainers.
Weeks 5 to 7	Production implementation, unit and integration tests, first pull request.
Weeks 8 to 9	Example flowgraph, documentation, review rounds.
Weeks 10 to 11	Full evaluation sweep across SNR and delay; compare with baseline and Doly et al.
Week 12 and final	Cleanup, final pull requests, final report and video.

7 Risks and how I will handle them

- The design may not match what the maintainers prefer. I will post the design before coding and treat feedback as part of the schedule.
- gr-radar is an out-of-tree module (from KIT), so a change there goes through that project rather than the main GNU Radio repository. I will confirm early which repository receives which change.
- The hypothesis in Section 4.2 may fail. If it does, a clear negative result with measurements, plus the reproduction flowgraphs and tests, is still a deliverable.
- Pull requests can stall for review. I will keep them small, answer feedback on the same branch, and report the real status, as in the M2 and M3 reports.
- Hardware validation is optional. All required results come from simulation.

8 About me and prior contribution

I am a student at Ahmedabad University working on this as the gsoc isac individual project.

My first contribution to GNU Radio is pull request #8233, a Required option for the GRC Parameter block (issue #5096). It changes four files with 30 added lines, was tested by hand on the no-GUI workflow and the Qt GUI workflow, and was signed off under the DCO. A maintainer asked me to follow the pull request template and converted it to a draft, and I updated the description to follow it. The pull request is not merged yet. I include it as evidence that I can work through the contribution process and respond to feedback.

9 Questions for mentors and the community

- Is a shared timing and channel estimate best added to gr-digital, to gr-radar, or as a small separate module?

- Is there an existing interface for carrying timing metadata with a stream that this should reuse?
- Which simulation channel models are accepted as reference for QA tests?
- Would the maintainers want the example flowgraph in the main repository or in an out-of-tree project?

References

1. C. Sturm and W. Wiesbeck, "Waveform design and signal processing aspects for fusion of wireless communications and radar sensing," Proceedings of the IEEE, vol. 99, no. 7, pp. 1236-1259, 2011.

2. F. Liu, C. Masouros, A. P. Petropulu, H. Griffiths, and L. Hanzo, "Joint radar and communication design: Applications, state-of-the-art, and the road ahead," IEEE Trans. Commun., vol. 68, no. 6, pp. 3834-3862, 2020.

3. F. Liu et al., "Integrated sensing and communications: Toward dual-functional wireless networks for 6G and beyond," IEEE J. Sel. Areas Commun., vol. 40, no. 6, pp. 1728-1767, 2022.

4. S. A. Doly, A. Chiriyath, W. Standage-Beier, and D. W. Bliss, "GNU Radio realization of Waveform Co-design for Joint Radar-Communications system using SDRs," Proceedings of the GNU Radio Conference, vol. 8, no. 1, 2023.

5. Communications Engineering Lab, Karlsruhe Institute of Technology, "gr-radar," GitHub, https://github.com/kit-cel/gr-radar

6. Q. Dai et al., "A Tutorial on MIMO-OFDM ISAC: From Far-Field to Near-Field," arXiv:2504.19091, 2025.

7. M. Vaezi, G. Amarasuriya Aruma Baduge, E. Ollila, and S. A. Vorobyov, "A Tutorial on AI-Empowered Integrated Sensing and Communications," IEEE Communications Surveys & Tutorials, vol. 28, 2026.

8. M. U. F. Qaisar et al., "The Role of ISAC in 6G Networks: Enabling Next-Generation Wireless Systems," IEEE Trans. Network Science and Engineering, vol. 13, 2026.

9. GNU Radio pull request #8233, https://github.com/gnuradio/gnuradio/pull/8233, and issue #5096, https://github.com/gnuradio/gnuradio/issues/5096.